

TECHNICAL IMPLEMENTATION SPECIFICATION

SlideForge AI

AI-Powered Presentation Builder

A Gamma.app-class, layout-driven presentation platform with AI generation, real-time editing, theme customization, and PPTX export.

Version 1.0 | April 2026 | Confidential

Table of Contents

1. Project Overview & Vision
2. Technology Stack Specification
3. System Architecture
4. Data Model & Database Schema
5. Core Module Specifications
6. AI Integration & Prompt Engineering
7. Layout & Composition Engine
8. Theme & Design Token System
9. PPTX Rendering Pipeline
10. Editor & Real-Time Collaboration
11. Authentication & Authorization
12. API Routes & Endpoints
13. Frontend Pages & Components
14. Testing Strategy
15. Deployment & DevOps
16. Implementation Phases & Milestones
17. Acceptance Criteria

1. Project Overview & Vision

1.1 Executive Summary

Build “SlideForge AI” — a production-grade, AI-powered presentation platform comparable to Gamma.app. The system enables users to generate fully-designed, layout-based presentations from natural language prompts, then customize every aspect through a real-time visual editor. Presentations are composed of vertical sections (not traditional fixed-size slides), each backed by a composable layout template with design-token-driven theming. The final output can be presented in-browser or exported as a pixel-perfect .pptx file.

1.2 Core Product Pillars

- **AI-First Generation:** Users describe their presentation in natural language; the AI produces a structured outline, selects layouts per section, generates content, and applies a cohesive theme — all in one shot.
- **Layout-Driven Composition:** Every section maps to a named layout template (e.g., “Hero with Image”, “Two-Column Stats”, “Timeline”). Layouts define slot positions, sizing constraints, and responsive breakpoints.
- **Deep Customizability:** Users can swap layouts, edit content inline (rich text via Tiptap), reorder sections via drag-and-drop, adjust theme tokens (colors, fonts, spacing), and override per-section styles.
- **Pixel-Perfect PPTX Export:** A dedicated renderer maps each layout and its content into PptxGenJS shapes, preserving fonts, colors, positioning, and media — producing files that open flawlessly in PowerPoint and Google Slides.
- **Collaboration-Ready Architecture:** Real-time multi-user editing with presence indicators, comments, and version history — built on a CRDT-friendly data model.

1.3 Target Users

- Startup founders creating pitch decks
- Marketing teams building campaign presentations
- Educators and trainers producing course materials
- Consultants generating client deliverables
- Anyone who needs beautiful presentations without design skills

2. Technology Stack Specification

2.1 Core Frameworks

Technology	Role & Version
Next.js 16 (Beta)	App Router, RSC, Server Actions, Middleware, API routes. The backbone of the application.
React 19	UI rendering with Suspense, useOptimistic, useTransition, and Server Components.
TypeScript (Strict)	Enforced across the entire codebase with strict: true, no implicit any, and path aliases.

2.2 Frontend & UI

Technology	Role & Configuration
Tailwind CSS 4	Utility-first styling with JIT compilation, custom design tokens mapped to CSS variables.
Radix UI	Accessible primitives: Dialog, Popover, Tooltip, DropdownMenu, Accordion, Tabs, Select, Slider.
Redux Toolkit + react-redux	Global state: active presentation, editor mode, selected section, undo/redo stack, theme overrides.
Lucide React + React Icons	Icon library for UI elements, toolbar actions, and section type indicators.
Tiptap (Headless)	Rich-text editor per content block: bold, italic, links, lists, code, markdown shortcuts, slash commands.
Recharts	Data visualization within presentation sections (bar, line, pie, area charts from user data).
Mermaid.js	Diagram rendering (flowcharts, sequence diagrams, mind maps) embeddable in sections.

2.3 Backend & Infrastructure

Technology	Role & Configuration
Prisma ORM	Type-safe database access with PostgreSQL. Schema defines Users, Presentations, Sections, Themes, Assets.
PostgreSQL	Primary data store. JSONB columns for flexible section content and layout metadata.

NextAuth.js v5	Authentication with Prisma adapter. Providers: Google, GitHub, email magic link, credentials.
Anthropic SDK	Primary AI provider for outline generation, content writing, and layout selection.
OpenAI SDK	Secondary AI provider, user-selectable. Used for image generation (DALL-E) and embeddings.
Google Generative AI	Tertiary AI provider (Gemini). Multimodal input for document-to-presentation conversion.
PptxGenJS	Server-side PPTX file generation. Maps layout slots to PowerPoint shapes with precise positioning.
Puppeteer	Headless browser for generating thumbnail previews and PDF export of presentations.

2.4 Dev & Tooling

Technology	Role
Vitest	Unit and integration testing with React Testing Library for component tests.
Zod	Runtime schema validation for API inputs, AI responses, theme configs, and form data.
PostHog	Product analytics: feature usage, funnel analysis, A/B testing, session replay.
Vercel Analytics	Web vitals, performance monitoring, and deployment analytics.
React Email + Resend	Transactional emails: welcome, share invitations, export completion, billing notifications.

3. System Architecture

3.1 High-Level Architecture

The application follows a layered architecture with clear separation of concerns. The Next.js App Router serves as both the frontend framework and the API layer, with Server Components for data-heavy pages and Client Components for interactive editors.

3.2 Architecture Layers

Layer	Responsibility	Key Technologies
Presentation Layer	React pages, layouts, and components rendered via App Router. SSR for SEO-critical pages, CSR for the editor.	Next.js 16, React 19, Tailwind CSS 4
Editor Layer	Interactive canvas: section selection, drag-and-drop reordering, inline Tiptap editing, toolbar controls, theme panel.	Redux Toolkit, Tiptap, Radix UI, DnD Kit
Composition Engine	Resolves layout templates, applies theme tokens, computes section dimensions, manages responsive breakpoints.	Custom TypeScript modules
AI Orchestration	Manages multi-step AI pipeline: outline → layout selection → content generation → image suggestions. Streams responses.	Anthropic/OpenAI/Google SDKs, Vercel AI SDK
Data Access Layer	Prisma models, repository pattern for CRUD, optimistic updates, real-time sync via WebSocket or polling.	Prisma, PostgreSQL, JSONB
Rendering Pipeline	Maps section data + layout template + theme tokens into PPTX shapes. Handles fonts, images, charts, tables.	PptxGenJS, Sharp, Puppeteer
Auth & Middleware	Session management, RBAC, rate limiting, CSRF protection, API key management for AI providers.	NextAuth.js v5, Middleware

3.3 Directory Structure

The project follows a modular structure aligned with the Next.js 16 App Router conventions:

Path	Purpose
/app	Next.js App Router: pages, layouts, API routes, loading/error states, metadata.
/app/(auth)	Auth route group: /login, /register, /verify with shared auth layout.
/app/(dashboard)	Dashboard route group: /presentations, /templates, /settings.
/app/editor/[id]	Main editor page with dynamic presentation ID routing.
/app/present/[id]	Full-screen presentation mode with keyboard navigation.
/app/api/ai	AI endpoints: /generate, /refine, /suggest-layout, /generate-image.
/app/api/export	Export endpoints: /pptx, /pdf, /thumbnail.
/components/editor	Editor components: Canvas, Toolbar, SectionList, ThemePanel, LayoutPicker.
/components/sections	Section renderers: HeroSection, TwoColumnSection, TimelineSection, etc.
/components/ui	Shared UI primitives built on Radix UI.
/lib/layouts	Layout template definitions: slot configs, constraints, responsive rules.
/lib/renderer	PPTX rendering logic: shape mappers, font handlers, image processors.
/lib/composer	Composition engine: section stacking, dimension calculation, breakpoints.
/lib/themes	Theme definitions: design tokens, presets, token resolver.
/lib/ai	AI pipeline: prompt templates, response parsers, provider abstraction.
/lib/validators	Zod schemas for all data structures.
/prisma	Database schema, migrations, and seed files.
/store	Redux store: slices for editor, presentation, theme, UI state.
/types	Shared TypeScript type definitions and interfaces.

4. Data Model & Database Schema

4.1 Entity Relationship Overview

The data model centers around the Presentation entity, which contains an ordered array of Sections. Each Section references a Layout template and stores its content as structured JSON. Themes are stored independently and can be shared across presentations.

4.2 Prisma Schema Models

4.2.1 User Model

Field	Type & Description
id	String @id @default(cuid()) — Primary key
name	String? — Display name
email	String @unique — Login identifier
emailVerified	DateTime? — Email verification timestamp
image	String? — Avatar URL
hashedPassword	String? — Bcrypt hash for credentials auth
plan	Enum(FREE, PRO, TEAM) @default(FREE) — Subscription tier
aiCreditsUsed	Int @default(0) — Monthly AI generation counter
presentations	Presentation[] — Relation to owned presentations
accounts	Account[] — OAuth provider accounts (NextAuth)
sessions	Session[] — Active sessions (NextAuth)

4.2.2 Presentation Model

Field	Type & Description
id	String @id @default(cuid())
title	String — Presentation title
description	String? — Optional description/summary
userId	String — FK to User (owner)
themeId	String — FK to Theme
sections	Section[] — Ordered section list
isPublic	Boolean @default(false) — Public sharing toggle

shareSlug	String? @unique — Vanity URL slug
thumbnail	String? — Auto-generated preview image URL
metadata	Json? — AI generation params, version info, export settings
createdAt / updatedAt	DateTime — Timestamps

4.2.3 Section Model

Field	Type & Description
id	String @id @default(cuid())
presentationId	String — FK to Presentation
layoutId	String — References a layout template key (e.g., “hero-split”)
order	Int — Sort position within presentation
content	Json — Structured content matching layout slot schema (see Section 5)
styleOverrides	Json? — Per-section theme overrides (background color, padding, etc.)
transitions	Json? — Entry/exit animation config
notes	String? — Speaker notes (plain text or markdown)
isHidden	Boolean @default(false) — Skip in presentation mode

4.2.4 Theme Model

Field	Type & Description
id	String @id @default(cuid())
name	String — Theme display name
userId	String? — FK to User (null = system preset)
isPreset	Boolean @default(false) — Built-in theme flag
tokens	Json — Full design token object (see Section 8)
preview	String? — Theme preview thumbnail URL

4.2.5 Asset Model

Field	Type & Description
id	String @id @default(cuid())
userId	String — FK to User
presentationId	String? — FK to Presentation (optional)

type	Enum(IMAGE, VIDEO, ICON, CHART_DATA, DOCUMENT) — Asset category
url	String — Storage URL (S3/Cloudflare R2)
filename	String — Original filename
mimeType	String — MIME type
sizeBytes	Int — File size
metadata	Json? — Dimensions, alt text, AI-generated description

5. Core Module Specifications

5.1 Layout System (/lib/layouts)

The layout system is the architectural backbone. Each layout is a TypeScript object that defines the visual structure of a section. Layouts declare named slots where content is placed, along with sizing constraints, positioning rules, and responsive behavior.

5.1.1 Layout Template Interface

Every layout must implement the following interface:

- **id:** string — Unique identifier (e.g., “hero-split-image”, “three-column-stats”)
- **name:** string — Human-readable name for the layout picker UI
- **category:** LayoutCategory — Enum: COVER, CONTENT, DATA, MEDIA, COMPARISON, TIMELINE, CTA, BLANK
- **description:** string — Short description for AI layout selection and user tooltips
- **thumbnail:** string — Path to preview image shown in the layout picker
- **slots:** SlotDefinition[] — Array of named content slots with type, position, and constraints
- **defaultContent:** Record<string, SlotContent> — Placeholder content for each slot
- **minHeight:** number — Minimum section height in pixels
- **maxHeight:** number | “auto” — Maximum height or auto-expand
- **supportedMediaTypes:** MediaType[] — Which media (image, video, chart, mermaid) this layout supports
- **pptxMapping:** PptxSlotMapping — How each slot maps to PPTX shape coordinates (see Section 9)

5.1.2 Required Layout Templates

Implement the following 18 layout templates at minimum, organized by category:

Category	Layout ID	Description & Slots
COVER	cover-centered	Centered title + subtitle + optional logo. Slots: title (h1), subtitle (text), logo (image), backgroundImage (image).
COVER	cover-split	Left text, right full-bleed image. Slots: title (h1), subtitle (text), description (richtext), image (image).
CONTENT	content-single	Full-width rich text block. Slots: heading (h2), body (richtext).

CONTENT	content-two-column	Side-by-side text columns. Slots: heading (h2), leftBody (richtext), rightBody (richtext).
CONTENT	content-with-image	Text left, image right (or reversed). Slots: heading (h2), body (richtext), image (image), imagePosition (config: left right).
DATA	stats-row	3-4 stat cards in a row. Slots: heading (h2), stats (array of {value, label, icon}).
DATA	chart-fullwidth	Heading + full-width chart. Slots: heading (h2), chartType (config), chartData (json), caption (text).
DATA	chart-with-text	Chart left, explanation right. Slots: heading (h2), chartType, chartData, body (richtext).
MEDIA	image-fullbleed	Full-bleed background image with overlay text. Slots: image (image), overlayTitle (h2), overlayText (text).
MEDIA	image-gallery	Grid of 2-6 images with captions. Slots: heading (h2), images (array of {src, caption}).
MEDIA	video-embed	Embedded video with description. Slots: heading (h2), videoUrl (url), body (richtext).
COMPARISON	comparison-two	Two-column comparison (e.g., before/after, pros/cons). Slots: heading (h2), leftTitle, leftItems (list), rightTitle, rightItems (list).
COMPARISON	feature-grid	Grid of feature cards (2x2 or 3x2). Slots: heading (h2), features (array of {icon, title, description}).
TIMELINE	timeline-horizontal	Horizontal timeline with milestones. Slots: heading (h2), milestones (array of {date, title, description}).
TIMELINE	timeline-vertical	Vertical timeline (scrollable). Same slots as horizontal.
CTA	cta-centered	Call-to-action with heading, description, and button. Slots: heading (h1), description (text), buttonText (text), buttonUrl (url).
CTA	cta-split	CTA left, image/form right. Slots: heading (h2), body (richtext), buttonText, image (image).

BLANK	blank-canvas	Empty section with a single richtext slot. Slots: content (richtext). For freeform content.
--------------	--------------	---

5.1.3 Slot Definition Schema

Each slot within a layout is defined with the following properties:

- **id:** string — Unique slot identifier within the layout (e.g., “title”, “leftBody”)
- **type:** SlotType — Enum: TEXT, RICHTEXT, HEADING, IMAGE, VIDEO, CHART, MERMAID, LIST, STATS, URL, CONFIG
- **label:** string — Human-readable label shown in the editor
- **required:** boolean — Whether the slot must have content for the layout to render
- **position:** { x: number, y: number, width: number, height: number } — Percentage-based bounding box (0-100)
- **constraints:** { minLength?, maxLength?, aspectRatio?, allowedFormats? } — Validation rules
- **responsive:** { mobile?: Partial<Position>, tablet?: Partial<Position> } — Breakpoint overrides
- **aiHint:** string — Instructions for the AI when generating content for this slot

5.2 Composition Engine (/lib/composer)

The composition engine is responsible for arranging sections vertically into a cohesive presentation. Unlike traditional slide-based tools, SlideForge uses a continuous vertical layout where each section can have a variable height. The composer calculates final dimensions, handles section spacing, manages page breaks for PPTX export, and coordinates responsive layout adjustments.

5.2.1 Composer Responsibilities

- `resolveSection(section, theme)`: Takes a Section record and Theme tokens, resolves the layout template, applies content to slots, computes final height, and returns a ResolvedSection with pixel-precise dimensions.
- `composePresentation(sections[], theme)`: Orchestrates all sections into a vertical stack, calculating cumulative Y offsets, inserting section gaps per theme tokens, and producing a ComposedPresentation ready for rendering.
- `splitForPptx(composedPresentation)`: Analyzes the vertical stack and determines optimal page break points for PPTX export. Each page maps to one PowerPoint slide. Sections taller than one slide height are split with continuation indicators.
- `computeResponsiveLayout(resolvedSection, viewport)`: Recalculates slot positions for different viewport sizes (mobile, tablet, desktop) using the responsive overrides in each slot definition.

5.3 Theme Engine (/lib/themes)

See Section 8 for the full theme and design token specification.

5.4 PPTX Renderer (/lib/renderer)

See Section 9 for the complete PPTX rendering pipeline specification.

6. AI Integration & Prompt Engineering

6.1 AI Pipeline Overview

The AI system follows a multi-step pipeline to transform a user's natural language prompt into a fully-structured, layout-aware presentation. Each step has a defined input, output, and Zod validation schema. The pipeline streams responses to the client for real-time feedback.

6.2 Pipeline Steps

Step	Input	Output
1. Outline Generation	User prompt (string), optional: tone, audience, length preference	Structured outline: array of section titles, key points per section, suggested section count (5-15)
2. Layout Selection	Generated outline, available layout templates with descriptions	Layout ID assigned per section, with rationale for each choice
3. Content Generation	Outline + assigned layouts, theme context, slot schemas per layout	Structured JSON content for every slot in every section: headings, body text, stats, chart data, image search queries
4. Image Suggestion	Content with image slots, presentation topic context	Unsplash/Pexels search queries or DALL-E generation prompts per image slot
5. Theme Suggestion	Presentation topic, tone, industry context	Recommended theme preset ID or custom token overrides
6. Refinement	Existing section content, user feedback (e.g., "make this more concise", "add more data")	Updated content for the specified sections

6.3 Prompt Templates

Each pipeline step uses a carefully engineered system prompt. Key principles:

- System prompts include the complete list of available layout templates with their slot schemas, so the AI understands exactly what content structure each layout expects.
- Content generation prompts are layout-aware: the AI receives the slot definitions for the assigned layout and must produce content that matches each slot's type and constraints.
- All AI responses must be valid JSON matching a Zod schema. The system prompt includes the exact JSON structure expected, with examples.
- Streaming is used for all generation steps. The client renders sections progressively as they arrive.

- Error recovery: if the AI produces invalid JSON, the system retries with a corrective prompt up to 3 times before falling back to a default template.

6.4 Provider Abstraction

Implement a provider abstraction layer (`lib/ai/providers/`) that normalizes the interface across Anthropic, OpenAI, and Google SDKs:

- AIProvider interface: `generateStructured<T>(systemPrompt, userPrompt, schema: ZodSchema<T>): AsyncGenerator<Partial<T>>`
- AnthropicProvider: Uses Claude 3.5/4 via the Anthropic SDK. Primary provider for text generation.
- OpenAIProvider: Uses GPT-4o via OpenAI SDK. Secondary text + DALL-E image generation.
- GoogleProvider: Uses Gemini via Google Generative AI SDK. Multimodal input (PDF/image to presentation).
- Provider selection: User preference stored in settings, fallback chain on rate limit or error.

6.5 AI Response Validation

Every AI response passes through strict Zod validation before being accepted:

- OutlineSchema: validates section count (3-20), title length, key point structure
- LayoutAssignmentSchema: validates that each assigned layoutId exists in the registry
- SectionContentSchema: per-layout validation ensuring all required slots have content matching type constraints
- ImageQuerySchema: validates search query format and aspect ratio hints
- ThemeSuggestionSchema: validates token structure matches ThemeTokens interface

7. Layout & Composition Engine (Detailed)

7.1 Section Rendering Flow

When a section is rendered in the browser editor, the following pipeline executes:

- **Layout Resolution:** The composer fetches the layout template by `layoutId` from the registry. If the layout doesn't exist (e.g., after a template update), fall back to "content-single".
- **Content Mapping:** Each key in the section's content JSON is matched to a slot ID in the layout. Unmapped content is ignored; empty required slots show placeholder content.
- **Theme Application:** The theme engine resolves design tokens (colors, fonts, spacing) and generates CSS custom properties scoped to the section element.
- **Height Calculation:** The composer calculates the section height based on content volume, minimum/maximum height constraints from the layout, and the current viewport width.
- **Responsive Adjustment:** Slot positions are recalculated if the viewport triggers a breakpoint override defined in the layout's responsive config.
- **React Rendering:** A React component for the layout type receives resolved props (content, theme tokens, dimensions) and renders the section DOM.

7.2 Drag-and-Drop Reordering

Section reordering uses a drag-and-drop library (DnD Kit or similar) with the following behavior:

- Drag handle visible on section hover (left edge grip icon)
- Smooth animated transitions during drag with ghost preview
- Drop zones highlighted between sections with insertion indicator
- Optimistic reorder in Redux state; confirmed via server action updating `Section.order` values
- Undo support: reorder is pushed to the undo stack

7.3 Section Operations

Each section supports the following operations via a context menu and toolbar:

- **Add Section Above/Below:** Opens the layout picker to insert a new section
- **Duplicate Section:** Deep clones content and `styleOverrides`, inserts below
- **Delete Section:** Soft delete with undo (30-second recovery window)
- **Change Layout:** Opens layout picker; existing content is remapped to the new layout's slots where slot types match
- **Hide/Show:** Toggles `isHidden` for presentation mode without deleting
- **Move to Position:** Direct reorder via numeric input
- **AI Refine:** Opens a chat panel to refine this section's content via AI

- Speaker Notes: Opens a text area below the section for presentation notes

8. Theme & Design Token System

8.1 Design Token Structure

Themes are defined as a structured set of design tokens that control every visual aspect of a presentation. The token structure is hierarchical and maps directly to CSS custom properties and PPTX style attributes.

8.1.1 Token Categories

Category	Tokens	PPTX Mapping
Colors	primary, secondary, accent, background, surface, textPrimary, textSecondary, textOnPrimary, textOnAccent, border, shadow	Shape fill, font color, background fill
Typography	fontFamily.heading, fontFamily.body, fontFamily.mono, fontSize (scale of 8 sizes), fontWeight (regular, medium, semibold, bold), lineHeight, letterSpacing	Font name, size (pt), bold/italic attributes
Spacing	sectionPadding (top, right, bottom, left), slotGap, innerPadding, sectionGap	Shape positioning offsets (inches)
Layout	maxWidth, borderRadius, shadowStyle (none, sm, md, lg), imageStyle (rounded, square, circle)	Shape dimensions, rounded corners
Effects	backgroundType (solid, gradient, image, pattern), gradientAngle, gradientStops[], overlayOpacity	Slide background fill

8.2 Theme Presets

Ship with at minimum 12 built-in theme presets:

- Minimal Light: Clean white background, Inter font, subtle gray accents. Professional default.
- Minimal Dark: Dark charcoal background, white text, blue accents. Modern tech feel.
- Corporate Blue: Navy/white palette, serif headings, structured and formal.
- Startup Bold: Vibrant gradients, large sans-serif type, energetic colors.
- Nature Green: Earth tones, organic shapes, soft shadows.
- Creative Neon: Dark background with neon accent colors, geometric patterns.
- Academic: Warm cream background, serif fonts, footnote-style captions.
- Pitch Deck: High contrast, data-friendly, chart-optimized color palette.

- Pastel Soft: Muted pastel palette, rounded corners, friendly aesthetic.
- Monochrome: Black and white only, strong typography hierarchy.
- Retro Warm: Warm oranges/browns, vintage-inspired type, textured backgrounds.
- Gradient Wave: Animated gradient backgrounds (static for PPTX), modern feel.

8.3 Theme Customization UI

The theme panel (right sidebar in editor) allows real-time customization:

- Preset Selector: Visual grid of theme preview thumbnails
- Color Palette Editor: Click any color token to open a color picker; changes preview instantly
- Font Selector: Dropdown for heading and body fonts from a curated Google Fonts list
- Spacing Controls: Sliders for padding and gap values with live preview
- Background Controls: Toggle between solid color, gradient (angle + stops), and image background
- Per-Section Override: Any section can override specific tokens; shown as “custom” badge in section list
- Export/Import: Themes can be exported as JSON and imported/shared between users

9. PPTX Rendering Pipeline

9.1 Pipeline Architecture

The PPTX export pipeline transforms the in-memory presentation model into a valid .pptx file. The pipeline runs server-side in a Next.js API route and uses PptxGenJS for file generation.

9.2 Pipeline Steps

- Step 1 — Composition Split: The composer's `splitForPptx()` method divides the vertical presentation into discrete slides. Each slide corresponds to one section (or a portion of a tall section). Target slide dimensions: 13.333" x 7.5" (widescreen 16:9).
- Step 2 — Theme Resolution: Theme tokens are resolved into PPTX-compatible values: hex colors, font names, point sizes, inch measurements.
- Step 3 — Layout-to-Shape Mapping: Each layout template's `pptxMapping` defines how slots translate to PPTX shapes. The mapping specifies: shape type (textbox, image, shape, table, chart), position (x, y in inches from top-left), size (w, h in inches), style overrides (fill, border, shadow, rotation).
- Step 4 — Content Rendering: For each slot, the content is rendered into the mapped PPTX shape. Rich text (from Tiptap) is converted to PptxGenJS text runs with formatting. Images are fetched, resized via Sharp, and embedded. Charts are rendered via PptxGenJS's native chart API.
- Step 5 — Asset Processing: All images are downloaded, optimized (Sharp: resize to max 1920px, compress to 85% quality JPEG), and embedded in the PPTX. SVG diagrams (Mermaid) are rasterized via Puppeteer.
- Step 6 — Slide Master: A slide master is generated from the theme tokens, setting default fonts, colors, and background for all slides.
- Step 7 — Assembly & Export: PptxGenJS assembles all slides, writes the .pptx buffer, and returns it as a downloadable file.

9.3 Rich Text to PPTX Conversion

Tiptap's JSON document format must be converted to PptxGenJS text runs. Implement a recursive converter:

- Paragraph nodes → new text line with paragraph-level formatting
- Text nodes → PptxGenJS TextProps with: bold, italic, underline, strike, color, fontSize, fontFace, hyperlink
- BulletList/OrderedList → PptxGenJS bullet/number formatting with indent levels
- Code blocks → Monospace font, shaded background rectangle behind text
- Links → PptxGenJS hyperlink property on the text run
- Headings → Larger font size, bold, mapped from theme's heading font

9.4 Image Handling

- Fetch images from URLs, cache on server filesystem during export
- Resize proportionally to fit slot dimensions (never stretch/distort)
- Convert all images to JPEG for smaller file size (except PNGs with transparency)
- Apply imageStyle from theme: rounded corners via clip path, circle crops
- Handle missing images gracefully: insert a branded placeholder with error message

10. Editor & Real-Time Collaboration

10.1 Editor Architecture

The editor is the primary user interface. It occupies the full viewport and consists of the following panels:

- Top Bar: Presentation title (editable), share button, export dropdown, present button, user avatar, undo/redo.
- Left Sidebar (collapsible): Section list with thumbnails, drag-and-drop reorder, add section button, section search/filter.
- Main Canvas (center): Vertical scroll of all sections rendered at their actual layout. Click any element to edit inline. Section hover shows action toolbar.
- Right Sidebar (collapsible): Context-sensitive panel. Default: Theme panel. On section select: layout picker + section settings. On text select: formatting toolbar.
- Bottom Bar: Zoom controls, keyboard shortcut help, AI generation progress indicator.

10.2 Inline Editing with Tiptap

Each text/richtext slot in a section renders a Tiptap editor instance configured for inline editing:

- Click to activate: Clicking a text element focuses the Tiptap instance and shows a floating toolbar.
- Floating Toolbar: Bold, italic, underline, strikethrough, link, heading level, alignment, color, bullet list, numbered list, code.
- Slash Commands: Type “/” to open a command palette: insert image, chart, divider, code block, emoji, AI refine.
- Markdown Shortcuts: Typing “# ” auto-converts to H1, “- ” to bullet, “1. ” to numbered list, etc.
- Placeholder Text: Empty slots show ghost placeholder text from layout’s defaultContent.
- Character Limits: Enforce maxLength from slot constraints with visual counter.

10.3 Undo/Redo System

Implement a comprehensive undo/redo stack in Redux:

- Every mutation (content edit, section reorder, theme change, layout swap) pushes a state snapshot or action descriptor to the undo stack.
- Undo (Ctrl+Z / Cmd+Z): Pops the last action from the undo stack, pushes it to the redo stack, and reverses the mutation.
- Redo (Ctrl+Shift+Z / Cmd+Shift+Z): Pops from redo stack, pushes to undo stack, re-applies the mutation.
- Stack size: 50 actions maximum. Oldest actions are discarded.
- Debounced text edits: Rapid typing is batched into single undo entries with a 500ms debounce.

10.4 Keyboard Shortcuts

Shortcut	Action
Ctrl/Cmd + S	Save presentation (auto-saves every 30s, this forces immediate save)
Ctrl/Cmd + Z / Shift+Z	Undo / Redo
Ctrl/Cmd + D	Duplicate selected section
Ctrl/Cmd + Backspace	Delete selected section
Ctrl/Cmd + Enter	Enter presentation mode
Ctrl/Cmd + E	Open export dialog
Ctrl/Cmd + K	Open command palette (search layouts, themes, actions)
Arrow Up/Down	Navigate between sections
Escape	Deselect current section / close panel
Ctrl/Cmd + Shift + L	Toggle left sidebar
Ctrl/Cmd + Shift + R	Toggle right sidebar

11. Authentication & Authorization

Implement NextAuth.js v5 with the Prisma adapter. Support the following authentication providers:

- Google OAuth 2.0: Primary social login. Scopes: email, profile.
- GitHub OAuth: For developer audience. Scopes: read:user, user:email.
- Email Magic Link: Passwordless email login via Resend transactional email.
- Credentials: Email + password with bcrypt hashing (10 salt rounds). Include forgot password flow.

11.1 Authorization Model

Role-based access control with the following levels:

- Owner: Full CRUD on their presentations. Can share, transfer ownership, delete permanently.
- Editor (future): Can edit content and layout but not delete or change sharing settings.
- Viewer (future): Read-only access to shared presentations.
- Admin (future): Platform-wide user management, template management, analytics access.

All API routes and Server Actions validate session via `getSession()` before processing requests. Unauthorized access returns 401. Access to specific presentations checks ownership via `userId` match.

12. API Routes & Endpoints

12.1 Presentation Endpoints

Method & Path	Description	Auth Required
GET /api/presentations	List user's presentations (paginated, sortable)	Yes
POST /api/presentations	Create new presentation (blank or from AI)	Yes
GET /api/presentations/[id]	Get full presentation with sections and theme	Yes (owner)
PATCH /api/presentations/[id]	Update title, description, theme, settings	Yes (owner)
DELETE /api/presentations/[id]	Soft delete presentation	Yes (owner)
POST /api/presentations/[id]/duplicate	Deep clone presentation	Yes

12.2 Section Endpoints

Method & Path	Description	Auth Required
POST /api/presentations/[id]/sections	Add section (specify layoutId, order, content)	Yes (owner)
PATCH /api/sections/[id]	Update section content, layout, order, overrides	Yes (owner)
DELETE /api/sections/[id]	Delete section	Yes (owner)
POST /api/sections/[id]/duplicate	Duplicate section within presentation	Yes (owner)
PATCH /api/presentations/[id]/reorder	Bulk reorder sections (array of {id, order})	Yes (owner)

12.3 AI Endpoints

Method & Path	Description	Auth Required
POST /api/ai/generate	Full pipeline: prompt → outline → layouts → content → theme. Streams response.	Yes
POST /api/ai/refine	Refine specific section(s) based on user feedback. Streams.	Yes

POST /api/ai/suggest-layout	Given section content, suggest best layout. Returns layoutId.	Yes
POST /api/ai/generate-image	Generate image via DALL-E or search Unsplash. Returns image URL.	Yes
POST /api/ai/outline-only	Generate outline without full content (for user review before committing).	Yes

12.4 Export Endpoints

Method & Path	Description	Auth Required
POST /api/export/pptx	Generate and download .pptx file. Body: {presentationId, options}.	Yes (owner)
POST /api/export/pdf	Generate PDF via Puppeteer. Body: {presentationId, options}.	Yes (owner)
POST /api/export/thumbnail	Generate preview thumbnail image. Returns image URL.	Yes (owner)

12.5 Theme & Asset Endpoints

Method & Path	Description	Auth Required
GET /api/themes	List available themes (presets + user custom)	Yes
POST /api/themes	Create custom theme from tokens	Yes
POST /api/assets/upload	Upload image/file asset. Returns URL.	Yes

13. Frontend Pages & Components

13.1 Page Routes

Route	Component	Description
<code>/</code>	LandingPage	Marketing page with hero, features, pricing, testimonials. SSR for SEO.
<code>/login</code>	LoginPage	Auth form with social providers and email/password.
<code>/register</code>	RegisterPage	Registration with email verification flow.
<code>/dashboard</code>	DashboardPage	Grid/list of user's presentations with search, sort, filter.
<code>/dashboard/templates</code>	TemplatesPage	Browse and preview starter templates. Click to create from template.
<code>/dashboard/settings</code>	SettingsPage	Profile, account, AI provider keys, billing, preferences.
<code>/editor/[id]</code>	EditorPage	Main presentation editor (see Section 10 for full spec).
<code>/present/[id]</code>	PresentationMode	Full-screen slideshow with keyboard navigation, progress bar, speaker notes view.
<code>/share/[slug]</code>	SharedPresentation	Public view of a shared presentation. No editing. Optional password protection.

13.2 Key Component Tree

Editor page component hierarchy:

- EditorPage → EditorLayout (manages panel visibility and sizing)
- EditorLayout → TopBar + LeftSidebar + MainCanvas + RightSidebar + BottomBar
- LeftSidebar → SectionList → SectionThumbnail[] (with DnD reorder)
- MainCanvas → SectionRenderer[] (maps section type to layout component)
- SectionRenderer → LayoutComponent (e.g., HeroSplitLayout, TwoColumnLayout) + SectionToolbar
- LayoutComponent → SlotRenderer[] (maps slot type to content component: TiptapSlot, ImageSlot, ChartSlot, etc.)
- RightSidebar → ThemePanel | LayoutPicker | SectionSettings (context-dependent)

- ThemePanel → PresetGrid + ColorPalette + FontSelector + SpacingControls + BackgroundEditor

14. Testing Strategy

All tests use Vitest as the test runner with React Testing Library for component tests.

14.1 Test Categories

Category	Scope	Minimum Coverage
Unit Tests	Pure functions: theme resolver, layout resolver, PPTX shape mapper, Tiptap-to-PPTX converter, Zod validators, AI response parsers	90% line coverage
Component Tests	React components: each layout component renders correctly with mock data, section toolbar actions fire correct Redux actions, theme panel controls update tokens	80% line coverage
Integration Tests	API routes: presentation CRUD, section operations, AI endpoints (with mocked AI responses), export endpoints (with mocked PptxGenJS)	80% line coverage
E2E Tests (future)	Critical user flows: create presentation via AI, edit section content, change theme, export PPTX, share presentation	Key flows covered

14.2 Test Data

Create comprehensive test fixtures:

- Mock presentations with every layout type represented
- Theme fixtures for all 12 presets
- AI response fixtures for each pipeline step (valid + invalid for error path testing)
- Image fixtures (placeholder PNGs of various aspect ratios)
- PPTX output validation: open generated .pptx, verify slide count, shape positions, text content

15. Deployment & DevOps

- Platform: Vercel (primary) or self-hosted Node.js with Docker.
- Database: Managed PostgreSQL (Neon, Supabase, or AWS RDS).
- File Storage: Cloudflare R2 or AWS S3 for uploaded assets and generated exports.
- Environment Variables: AI API keys, database URL, NextAuth secret, S3 credentials, Resend API key, PostHog project key. All stored in Vercel env or .env.local.
- CI/CD: GitHub Actions — lint, type-check, test, build on every PR. Auto-deploy to Vercel on merge to main.
- Monitoring: Vercel Analytics for web vitals, PostHog for product analytics, Sentry for error tracking.

16. Implementation Phases & Milestones

Phase 1: Foundation (Weeks 1-3)

- Project scaffolding: Next.js 16, TypeScript, Tailwind CSS 4, ESLint, Prettier
- Prisma schema + PostgreSQL setup + seed data
- NextAuth.js v5 with Google + GitHub + email providers
- Redux store setup with editor, presentation, and theme slices
- Base UI component library (Radix UI wrappers)
- Landing page + auth pages + dashboard shell

Phase 2: Layout Engine (Weeks 4-6)

- Layout template registry with all 18 layout definitions
- Composition engine: resolveSection, composePresentation, height calculation
- Section renderer: maps layout templates to React components
- Slot renderer: content components for each slot type (text, image, chart, list)
- Theme engine: token resolver, CSS variable generation, preset loading
- All layout components rendering correctly with mock data + tests

Phase 3: Editor (Weeks 7-9)

- Editor page layout with all panels (top bar, sidebars, canvas, bottom bar)
- Tiptap integration for all text/richtext slots
- Inline editing with floating toolbar
- Section list with drag-and-drop reordering
- Layout picker panel
- Theme customization panel with live preview
- Undo/redo system
- Keyboard shortcuts
- Auto-save with debounced persistence

Phase 4: AI Integration (Weeks 10-12)

- AI provider abstraction layer
- Full generation pipeline: outline → layouts → content → images → theme
- Streaming response rendering with progressive section display
- AI refinement for individual sections
- Zod validation for all AI responses with error recovery

- Generation progress UI with cancel support
- Rate limiting and AI credit tracking

Phase 5: Export & Presentation (Weeks 13-15)

- PPTX rendering pipeline with PptxGenJS
- Tiptap-to-PPTX text converter
- Image processing pipeline (fetch, resize, embed)
- Chart rendering in PPTX (Recharts data → PptxGenJS charts)
- Mermaid diagram rasterization via Puppeteer
- PDF export via Puppeteer
- Presentation mode (full-screen, keyboard navigation, speaker notes)
- Thumbnail generation for dashboard previews

Phase 6: Polish & Launch (Weeks 16-18)

- Starter templates (10+ pre-built presentations covering common use cases)
- Sharing: public links, password protection, embed code
- Analytics integration (PostHog + Vercel Analytics)
- Email notifications (welcome, share invites, export ready)
- Performance optimization: lazy loading, image optimization, bundle splitting
- Comprehensive test suite completion
- Documentation: README, API docs, contributing guide
- Production deployment and monitoring setup

17. Acceptance Criteria

The implementation is considered complete when ALL of the following criteria are met:

17.1 Functional Requirements

- A user can sign up, log in (social + email), and manage their account.
- A user can create a new presentation from a natural language prompt, and the AI generates a complete multi-section presentation with appropriate layouts and content within 30 seconds.
- All 18 layout templates render correctly in the browser with proper slot positioning and responsive behavior.
- A user can edit any text/content slot inline using the Tiptap editor with full rich text formatting.
- A user can drag-and-drop sections to reorder them, and the change persists on save.
- A user can change the layout of any section, and existing content remaps to compatible slots.
- A user can customize the theme (colors, fonts, spacing, background) and see changes reflected in real time across all sections.
- A user can export the presentation as a .pptx file that opens correctly in Microsoft PowerPoint and Google Slides with accurate layout, fonts, colors, and images.
- A user can present their presentation in full-screen mode with keyboard navigation (arrow keys, Escape to exit).
- A user can share a presentation via a public link.

17.2 Non-Functional Requirements

- Performance: Editor loads in < 2 seconds. Section rendering < 100ms. PPTX export < 15 seconds for a 20-section presentation.
- Reliability: AI generation includes retry logic with fallback templates. No data loss on browser crash (auto-save).
- Accessibility: All UI controls meet WCAG 2.1 AA. Keyboard navigable. Screen reader compatible (Radix UI).
- Security: All inputs sanitized. CSRF protection. Rate limiting on AI and export endpoints. No secret exposure in client bundle.
- Code Quality: TypeScript strict mode. No any types. ESLint clean. Vitest coverage meets minimums per category.
- SEO: Landing page and shared presentations are server-rendered with proper meta tags and Open Graph images.

End of Technical Specification — SlideForge AI v1.0